

Unit 3

SYSTEM ANALYSIS IN OOAD

3.1. **Introduction To OOAD:**

Object-Oriented Analysis and Design (OOAD) is a way to design software by thinking of everything as objects similar to real-life things.

3.1.1. **Important Aspects of OOAD:**

Below are some important aspects of OOAD:

1. **Object-Oriented Programming:** In this the real-world items are represented/mapped as software objects with attributes and methods that relate to their actions.
2. **Design patterns:** are used by OOAD to help developers in building software systems that are more efficient and maintainable.
3. **UML diagrams** are used in OOAD to represent the different components and interactions of a software system.
4. **Use Cases:** OOAD uses use cases to help developers understand the requirements of a system and to design software systems that

3.1.1.1. **Object-Oriented Analysis:**

- Object-Oriented Analysis (OOA) is the process of understanding and analyzing the system requirements by looking at the problem scenario in terms of objects.
- These objects represent real-world entities or concepts that are relevant to the system being developed.
- During OOA, the goal is to identify the objects, their attributes, behaviors, and relationships, without focusing on how the system will be implemented.

3.1.1.2. **Object-Oriented Design:**

- In the object-oriented software development process, the analysis model, which is initially formed through object-oriented analysis (OOA), undergoes a transformation during object-oriented design (OOD) i.e implementation of the conceptual model developed in OOA.
- This evolution is crucial because it shapes the analysis model into a detailed design model.
- Furthermore, as part of the object-oriented design process, it is essential to define specific aspects:

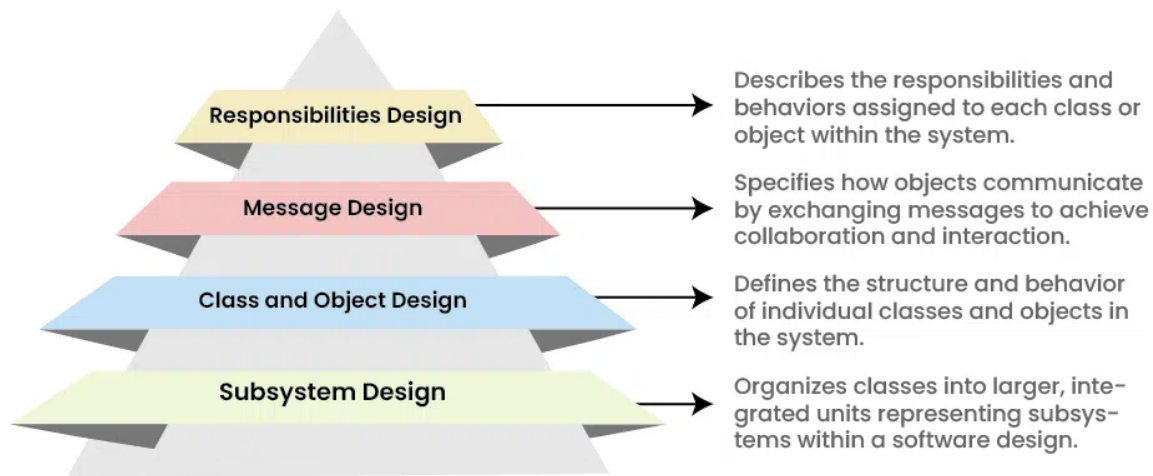
Data Organization of Attributes:

- OOD involves specifying how data attributes are organized within the objects.
- This includes determining the types of data each object will hold and how they relate to one another.

Procedural Description of Operations:

- OOD requires a procedural description for each operation that an object can perform.
- This involves detailing the steps or processes involved in carrying out specific tasks.

Below diagram shows a design pyramid for object-oriented systems. It is having the following four layers.



The Object Oriented Design Pyramid



The Subsystem Layer: It represents the subsystem that enables software to achieve user requirements and implement technical frameworks that meet user needs.

The Class and Object Layer: It represents the class hierarchies that enable the system to develop using generalization and specialization. This layer also represents each object.

The Message Layer: This layer deals with how objects interact with each other. It includes messages sent between objects, method calls, and the flow of control within the system.

The Responsibilities Layer: It focuses on the responsibilities of individual objects. This includes defining the behavior of each class, specifying what each object is responsible for, and how it responds to messages.

3.1.3. Benefits of Object-Oriented Analysis and Design (OOAD)

- It increases the modularity and maintainability of software by encouraging the creation of tiny, reusable parts that can be combined to create more complex systems.
- It provides a high-level, abstract representation of a software system, making understanding and maintenance easier.
- It promotes object-oriented design principles and the reuse of objects, which lowers the amount of code that must be produced and raises the quality of the program.
- Software engineers can use the same language and method that OOAD provides to communicate and work together more successfully in groups.
- It can assist developers in creating scalable software systems that can adapt to changing user needs and business demands over time.

3.1.4. Challenges of Object-Oriented Analysis and Design (OOAD)

- Because objects and their interactions need to be carefully explained and handled, it might complicate a software system.
- Because objects must be instantiated, managed, and interacted with, this may result in additional overhead and reduce the software's speed.
- For beginner software engineers, OOAD might have a challenging learning curve

- It can be a time-consuming process that involves significant upfront planning and documentation.
- OOAD can be more expensive than other software engineering methodologies

3.1.4. Real world applications of Object-Oriented Analysis and Design(OOAD)

Some examples of OOAD's practical uses are listed below:

1. **Banking Software:** In banking systems, OOAD is frequently used to simulate complex financial transactions, structures, and customer interactions. Designing adaptable and reliable financial apps is made easier by OOAD's modular and scalable architecture.
2. **Electronic Health Record (EHR) Systems:** Patient data, medical records, and healthcare workflows are all modelled using OOAD. Modular and flexible healthcare apps that may change to meet emerging requirements can be made through object-oriented principles.
3. **Flight Control Systems:** OOAD is crucial in designing flight control systems for aircraft. It helps model the interactions between different components such as navigation systems, sensors, and control surfaces, ensuring safety and reliability.
4. **Telecom Billing Systems:** In the telecom sector, OOAD is used to model and build billing systems. It enables the modular and scalable modeling of complex subscription plans, invoicing rules, and client data.
5. **Online Shopping Platforms:** E-commerce system development frequently makes use of OOAD. Product catalogues, user profiles, shopping carts, and payment procedures are all modelled, which facilitates platform maintenance and functionality expansion.

Advantages of Object-Oriented Analysis:

1. **Modularity and Reusability:**
Systems are broken into objects, which promotes modularity.
Code and design elements can be reused across different parts of the system or in other projects.
2. **Better Mapping to Real-World Concepts:**
Objects represent real-world entities, making it easier to understand and model problems.
3. **Improved Maintainability:**
Encapsulation ensures that changes in one part of the system have minimal impact on others.
4. **Encourages Clear Structure and Design:**
Promotes hierarchical and organized design through classes and inheritance.
5. **Supports Iterative Development:**
Facilitates incremental development and refinement of system models.
6. **Enhanced Collaboration:**
Clearly defined objects and interfaces improve team communication and parallel development.
7. **Easier Transition to Object-Oriented Programming (OOP):**
Since the analysis and design are object-oriented, it aligns naturally with OOP languages like Java, C++, Python, etc.

Disadvantages of Object-Oriented Analysis:

- 1. Steep Learning Curve:**
May be difficult for beginners or those used to procedural or functional paradigms.
- 2. Overhead in Simple Systems:**
For small or simple projects, OOA can add unnecessary complexity.
- 3. Complex Relationships:**
Inheritance and polymorphism can lead to complicated class hierarchies if not managed properly.
- 4. Requires High Initial Effort:**
Defining and refining objects and classes takes more upfront effort during the analysis phase.
- 5. Tool and Skill Dependency:**
Requires familiarity with object-oriented modeling tools (e.g., UML) and concepts.
- 6. Performance Overhead:**
Object-oriented systems can introduce performance issues due to abstraction layers and dynamic binding.

Advantages of Object-Oriented Design:

- 1. Modularity:**
The system is divided into discrete objects, making the code more organized and manageable.
- 2. Reusability:**
Objects and classes can be reused in different programs, reducing development time and cost.
- 3. Maintainability and Scalability:**
Changes can be made to individual objects with minimal impact on the rest of the system, making the system easier to update and scale.
- 4. Improved Code Clarity and Structure:**
Concepts like encapsulation, inheritance, and polymorphism make code more understandable and reduce redundancy.
- 5. Real-World Modeling:**
Objects can represent real-world entities, making systems easier to conceptualize and design.
- 6. Parallel Development:**
Teams can develop different classes and objects simultaneously, improving productivity and collaboration.
- 7. Easier Debugging and Testing:**
Modular structure and encapsulated data help isolate issues and simplify testing.

Disadvantages of Object-Oriented Design:

- 1. Complexity for Small Projects:**
For simple systems, OOD may introduce unnecessary layers of abstraction and overhead.
- 2. Steep Learning Curve:**
Requires a solid understanding of object-oriented principles, which can be difficult for beginners.
- 3. More Planning Required:**

Requires detailed design and planning upfront to define all necessary objects and relationships.

4. Performance Overhead:

Additional abstraction and dynamic method binding can lead to slower performance compared to procedural designs.

5. Difficulty in Design Decisions:

Choosing the right classes, inheritance hierarchies, and relationships can be challenging and may require redesign later.

6. Tool Dependency:

Often requires specialized tools and frameworks, which may have their own learning curves and limitations.

3.2. System Analysis and Design:

- In the OOAD context, System Analysis focuses on understanding what the system should do by analyzing the problem domain through object-oriented concepts like classes, objects, use cases, and interactions.

Key Objectives of System Analysis in OOAD:

- Understand the Problem Domain:
- Identify the real-world entities, actors, and processes involved.
- Focus on requirements gathering and modeling the system behavior.
- Identify Objects and Classes.
- Define Use Cases.
- Model System Behavior.
- Establish System Boundaries.
- Clearly define what is inside and what is outside the scope of the system.

Key Components of System Analysis in OOAD:

Component	Description
Actors	External users or systems interacting with the system.
Use Cases	Functionalities or services provided by the system.
Class Identification	Finding classes based on nouns in problem description.
Relationships	Associations, generalizations, and aggregations between classes.
Object Interaction	Use of Sequence Diagrams or Collaboration Diagrams to model how objects communicate.
State Modeling	Using State Diagrams to represent object states and transitions.

3.3. Requirements in Object-Oriented Systems:

- Requirements engineering is a critical first step in the development of object-oriented systems.
- It involves identifying, documenting, and managing the needs that the system must satisfy.
- Requirements define what a system should do, not how it should do it
- Requirements serve as a contract between stakeholders and development team
- In object-oriented systems, requirements should align with object-oriented principles

Types of Requirements in Object-Oriented Systems**3.3.1. Functional Requirements**

- Functional requirements define what a system must do and what its features and functions are. They specify the system's behavior under specific conditions.

Characteristics of Functional Requirements:

- Describe specific behaviors or functions
- Define inputs, processes, and outputs
- Are typically expressed as "The system shall..."
- Map directly to use cases in object-oriented systems
- Can be validated through testing

Examples:

- The system shall allow users to create new customer accounts
- The system shall authenticate users before granting access
- The system shall calculate total order amounts including applicable taxes
- The system shall send email notifications when orders are processed

3.3.2. Non-functional Requirements

- Non-functional requirements describe the general properties or qualities of a system rather than its specific behaviors. They define constraints and quality attributes.

Types of Non-functional Requirements:

- Performance
- Response time
- Security
- Usability
- User satisfaction
- Reliability
- Availability
- Fault tolerance
- Recoverability
- Scalability
- Ability to handle growth
- Maintainability
- Modularity
- Testability

Examples:

- The system shall respond to user queries within 2 seconds under normal load
- The system shall be available 99.9% of the time during business hours
- The system shall support at least 1000 concurrent users
- The system shall comply with GDPR data protection regulations

3.3.3. Relationship Between Requirements Types in Object-Oriented Systems

In object-oriented systems, functional requirements typically map to:

- Object behaviors (methods)
- System services
- Business processes

Non-functional requirements typically affect:

- Class structure
- Interface design
- System architecture
- Implementation decisions

3.4. Requirements Modeling in Object-Oriented Analysis and Design (OOAD)

- Requirements modeling in OOAD involves creating visual representations of requirements to better understand and communicate system needs.

3.4.1. Purpose of Requirements Modeling in OOAD

- Establish a common understanding between users and developers
- Document system requirements in a standardized notation
- Facilitate analysis and validation of requirements
- Provide a foundation for design and implementation
- Support traceability from requirements to implementation

3.4.2. Key Requirements Modeling Techniques in OOAD**1. Use Case Modeling**

- Identifies actors and their interactions with the system
- Captures functional requirements in user-centric terms
- Provides context for system operations

2. Class Modeling (for domain analysis)

- Identifies key domain concepts and their relationships
- Creates an initial conceptual model of the problem domain
- Forms the basis for later design-level class diagrams

3. Activity Modeling

- Represents workflows and business processes
- Captures sequential and parallel activities
- Models decision points and alternative flows

4. State Modeling

- Captures object lifecycle and state changes
- Models system responses to events
- Identifies valid state transitions

3.5. Use Case Diagrams for Modeling System Requirements

- Use Case Diagrams are a key UML diagram type for capturing and visualizing functional requirements in object-oriented systems.

3.5.1. Elements of Use Case Diagrams

1. Actors

- External entities that interact with the system
- Can be humans, other systems, or devices
- Represented by stick figures in UML notation
- Named with noun phrases describing their role

2. Use Cases

- Units of functionality that provide value to an actor
- Represented by ovals in UML notation
- Named with verb phrases describing the action
- Should be atomic, cohesive, and meaningful to users

3. System Boundary

- Represented by a rectangle containing use cases
- Separates what's inside the system from what's outside
- Helps clarify system scope

4. Relationships

- Association: Basic relationship between actor and use case
- Include: One use case includes the functionality of another
- Extend: One use case may extend another under certain conditions
- Generalization: One actor/use case inherits from another

3.5.3. Tips for how to Use Use Case Diagrams

- Focus on user goals, not system functions
- Keep diagrams simple and focused
- Include only the most important use cases in a single diagram
- Use consistent naming conventions
- Avoid excessive relationships between use cases
- Ensure each use case provides clear value to at least one actor

3.6. Creating Use Case Descriptions for Functional Requirements:

Use Case Descriptions provide detailed textual explanations of each use case identified in the Use Case Diagram.

3.6.1. Structure of a Use Case Description

A comprehensive use case description typically includes:

- **Use Case Name:** A clear, action-oriented name (verb phrase)
- **ID:** Unique identifier for the use case
- **Description:** Brief summary of what the use case accomplishes
- **Actors:** Primary and secondary actors involved
- **Preconditions:** Conditions that must be true before the use case can start
- **Postconditions:** Conditions that will be true after successful completion

- **Main Flow:** Step-by-step sequence of events in the normal scenario
- **Alternative Flows:** Variations from the main flow
- **Exception Flows:** Error conditions and how they are handled
- **Business Rules:** Constraints or rules that must be enforced
- **Non-functional Requirements:** Performance, security, etc. specific to this use case
- **Relationships:** Include, extend, or generalization relationships with other use cases

Examples: Library Management System

